■ AI TRADING BOT

Build Your Own Trading Bot

Own Logic • Claude Prompts • Backtesting
Broker API Setup • Trading Basics — All in One Guide

■ Strategy Design	■ Claude Prompts	■ Broker API Setup
■ Backtesting	■ Live Bot Logic	■■ Risk Management

Guide by @levelupsam • Built entirely with Claude AI

■ WHAT'S IN THIS GUIDE

PART	TOPIC	WHAT YOU'LL LEARN
0	Trading Basics First	Markets, order types, risk — before touching any code
1	How a Trading Bot Works	Architecture, signal → order flow, paper vs live
2	Get Your Broker API Key	Upstox, Zerodha, Angel One — step by step
3	Design Your Own Strategy	RSI, EMA, ORB, VWAP — think before you code
4	Build Prompts for Claude	Exact prompts to generate every piece of bot code
5	Fetch & Prepare Market Data	Pulling candles, cleaning data, saving CSVs
6	Backtest Your Strategy	Simulate trades, measure performance, iterate
7	Read Your Results	Win rate, drawdown, Sharpe — what matters
8	Go Live — Paper Trade First	Connect broker, paper trade, then real money
9	Risk & Safety Rules	Position sizing, stop losses, API safeguards
10	Troubleshooting & Next Steps	Common errors, improvements, what to study next

PART 0 — Trading Basics First

Do not skip this section. Most people rush into bots without understanding markets. That is how they lose money fast. Spend one week on these basics before touching any code.

What is a financial market?

A stock exchange (NSE/BSE in India) is a marketplace where buyers and sellers trade shares of companies. Prices change every second based on supply and demand. Your bot will try to predict short-term price movements and place buy/sell orders automatically.

Key terms every bot builder must know

Term	What it means	Why it matters for bots
OHLCV	Open, High, Low, Close, Volume — the 5 values in every candle	Your bot reads these to make decisions
Candle / Bar	One time period of price data (e.g. 1 minute)	1-min candles = 375 candles per trading day
Long	Buying, expecting price to rise	Buy signal → go long
Short	Selling first, expecting price to fall (F&O only)	Sell signal → short (or just exit long)
Stop Loss	Auto-exit if trade goes wrong by X amount	Protects your capital — MANDATORY
Target	Auto-exit when profit reaches X amount	Locks in gains
Slippage	Difference between expected and actual fill price	Backtests rarely model this — be careful
Liquidity	How easily a stock is bought/sold without moving price	Trade only high-volume stocks in bots
Intraday	Buy and sell within the same day — no overnight position	Safer for bots — no overnight risk
Square Off	Closing all open positions	Bots must auto square-off by 3:20 PM

Order types your bot will use

Order Type	Description	When to use
Market Order	Execute immediately at best available price	Exits, urgent entries — beware slippage
Limit Order	Execute only at your specified price or better	Entries — more precise, may not fill
SL-M Order	Stop-loss market — triggers at price, fills at market	Stop losses in fast markets
SL Order	Stop-loss limit — triggers at price, fills at limit	When you want price control on SL

Indian Market Hours (NSE)

Session	Time	Notes
Pre-market	9:00 – 9:15 AM	Price discovery, avoid trading bots here
Regular Market	9:15 AM – 3:30 PM	375 minutes of 1-min candles
Bot Trading Window	9:15 AM – 3:20 PM	Stop 10 min early to allow square-off
Post-market	3:40 – 4:00 PM	Closing session, bots should be OFF
Futures & Options	9:15 AM – 3:30 PM	F&O specific — more complex

PART 1 — How a Trading Bot Works

A trading bot is just a Python script that runs in a loop, reads market data, applies your logic, and places orders automatically. There is no magic. Understanding this loop is everything.

The Bot Loop — Every Second / Every Candle Close

Step	What Happens	Code does this
1	Fetch latest candle from broker API	API call → OHLCV data
2	Calculate indicators (EMA, RSI, VWAP)	pandas / ta library
3	Check your strategy conditions	if/else logic
4	Decision: Buy / Sell / Hold	Signal generated
5	Place order via broker API	API call → order placed
6	Track position & P&L	Variables updated
7	Check stop loss / target / time exit	Risk management logic
8	Wait for next candle → repeat	time.sleep() loop

Paper Trading vs Live Trading

	Paper Trading	Live Trading
Money at risk	None — simulated	Real money
API used	Same real API calls	Same real API calls
Orders placed	Logged but not sent (or sandbox)	Actually executed
Purpose	Test bot logic in real-time	Profit/loss is real
When to use	ALWAYS start here — minimum 2 weeks	Only after paper proves strategy works

PART 2 — Get Your Broker API Key

You need an API key to connect your Python bot to the broker. The API lets your code fetch live prices, place orders, and check your account — all without logging into the website.

Which Broker Should You Choose?

Broker	API Name	Best For	Monthly Cost	Difficulty
Upstox	Upstox API v3	Beginners, free API	Free	★★■■■
Zerodha	Kite Connect	Best documentation	■2,000/mo	★★★■■
Angel One	SmartAPI	Free, good for algos	Free	★★■■■
Fyers	Fyers API v3	Options traders	Free	★★★■■
Dhan	Dhan API	Modern, good support	Free	★★■■■

Upstox API Setup — Step by Step (Recommended for Beginners)

Step 1: Create Account

- Go to upstox.com and sign up
- Complete KYC with PAN + Aadhaar
- Takes 1–2 business days to activate

Step 2: Developer Portal

- Go to developer.upstox.com
- Log in with your Upstox credentials
- Click 'Create New App'
- App Name: anything (e.g. MyTradingBot)
- Redirect URI: https://127.0.0.1 ← type exactly
- Submit → copy API Key + API Secret to Notepad

Step 3: Daily Access Token

- Upstox tokens expire every 24 hours
- You must run a token script every morning before trading
- Give Claude the prompt below to build this script

■ Claude Prompt — Build Your Token Script

```
I have an Upstox API Key: [YOUR KEY] and API Secret: [YOUR SECRET].
The redirect URI is https://127.0.0.1.
Write a Python script that:
1. Opens the Upstox login URL in my browser automatically
2. Asks me to paste the redirect URL after I log in
3. Extracts the auth code from the URL
4. Fetches and prints my access token
5. Saves the token to a file called access_token.txt
Add error handling and clear instructions for the user.
```

Angel One SmartAPI Setup — Free Alternative

- Sign up at angelone.in and complete KYC
- Go to smartapi.angelbroking.com → Create App
- App type: select 'Personal'
- Copy your API Key (no secret needed — uses TOTP)
- Enable TOTP in your Angel One app for 2FA
- Tokens are session-based — regenerate each login

■ Claude Prompt — Angel One Token Script

```
I use Angel One SmartAPI. My API Key is [KEY], Client ID is [ID],
Password is [PASS], and TOTP Secret is [TOTP].
Write a Python script that logs in using the smartapi-python library,
fetches my session token, prints it, and saves it to access_token.txt.
Handle login errors and TOTP generation automatically.
```

PART 3 — Design Your Own Strategy

The most important step is thinking clearly about your strategy rules BEFORE coding. Write them in plain English first. Claude will then turn them into code. If you can't explain your strategy in plain sentences, you don't understand it well enough yet.

The 4 Questions Every Strategy Must Answer

Question	Example Answer (EMA Crossover)
When do I ENTER a trade?	When the 9 EMA crosses above the 21 EMA on a 1-min chart
When do I EXIT for profit?	When the 9 EMA crosses back below the 21 EMA
When do I EXIT to cut loss?	When price falls 0.5% below my entry price (stop loss)
How much do I risk per trade?	Max 1% of total capital on any single trade

5 Starter Strategies — Plain English Rules

A — EMA Crossover	
• Entry: 9 EMA crosses above 21 EMA (bullish crossover)	
• Exit profit: 9 EMA crosses back below 21 EMA	
• Exit loss: Price drops 0.5% below entry	
• Best on: Trending stocks, 1-min or 5-min charts	
• Beginner friendly: YES — simple and popular	

B — RSI Reversal	
• Entry: RSI drops below 30 (oversold) then bounces back above 32	
• Exit profit: RSI rises above 65 OR price hits +0.8% target	
• Exit loss: RSI drops below 25 (stop loss) OR -0.4% from entry	
• Best on: Range-bound stocks, sideways markets	
• Beginner friendly: YES — RSI is easy to understand	

C — Opening Range Breakout (ORB)	
• Setup: Note the HIGH and LOW between 9:15–9:30 AM	
• Entry (Long): Price breaks ABOVE the range high with volume	
• Entry (Short): Price breaks BELOW the range low with volume	
• Exit: End of day OR 1.5x range size as target, 0.5x range as SL	
• Best on: High-volatility stocks, budget/result days	
• Beginner friendly: YES — very popular in India	

D — VWAP Strategy	
• Entry: Price crosses above VWAP from below (bullish)	
• Exit profit: Price = VWAP + 0.5% OR falls back below VWAP	
• Exit loss: Price drops 0.3% below entry	

• Best on: Index ETFs (NIFTY BEES), large caps	
• Beginner friendly: MODERATE — VWAP resets daily	

E — Supertrend Strategy	
• Entry: Supertrend indicator flips to bullish (green) signal	
• Exit: Supertrend flips back to bearish (red)	
• Stop loss: Built into Supertrend level	
• Best on: Trending markets, 5-min to 15-min charts	
• Beginner friendly: YES — clear buy/sell signals	

PART 4 — Build Your Bot with Claude Prompts

You do not need to know how to code. You need to know how to give Claude clear, detailed instructions. Below are proven prompts for every step of building your bot. Copy, fill in the brackets, and run.

The Golden Rules of Prompting Claude for Code

- Be specific — vague prompts get vague code
- Describe your exact file names and column names
- Tell Claude which libraries to use (pandas, ta, upstox-python-sdk)
- Ask for comments in the code so you can understand it
- If something breaks, paste the exact error message to Claude
- Ask Claude to explain each section in plain English at the end

Prompt Library — Copy and Use

■ Prompt 1 — Install Everything

```
I am building a Python trading bot for Indian markets using Upstox.
Tell me exactly what to install with pip. I need: Upstox SDK,
pandas, numpy, ta (technical analysis), matplotlib, requests,
schedule (for running code at specific times), and dotenv for
storing API keys safely. Give me one pip install command.
```

■ Prompt 2 — Fetch Live 1-Minute Candle

```
Using Upstox API v3 and Python, write a function called
get_latest_candle(instrument_key, access_token) that fetches
the most recent 1-minute OHLCV candle for a given stock.
Return it as a pandas DataFrame with columns: timestamp,
open, high, low, close, volume. Add error handling.
```

■ Prompt 3 — Calculate Indicators

```
I have a pandas DataFrame called df with columns: timestamp,
open, high, low, close, volume. Write a function called
add_indicators(df) that adds these columns:
ema_9 (9-period EMA), ema_21 (21-period EMA),
rsi_14 (14-period RSI), vwap (daily VWAP reset each day).
Use the 'ta' library. Return the updated DataFrame.
```

■ Prompt 4 — Strategy Signal Logic

```
Write a function called generate_signal(df) that takes my
DataFrame with indicators and returns 'BUY', 'SELL', or 'HOLD'.
Rules: Return BUY when ema_9 crosses above ema_21 on the
latest candle (was below on previous candle, now above).
Return SELL when ema_9 crosses below ema_21.
Return HOLD otherwise. Add clear comments.
```

■ Prompt 5 — Place an Order (Upstox)

```
Using Upstox API v3 and Python, write a function called
place_order(access_token, instrument_key, transaction_type,
quantity) that places a market intraday order.
transaction_type is 'BUY' or 'SELL'. Print the order ID
if successful. Handle API errors and print them clearly.
```

■ Prompt 6 — Position & P&L Tracker

```
Write a Python class called PositionTracker that tracks:
- entry_price, entry_time, quantity, side (BUY/SELL)
- current unrealized P&L given current price
- whether stop loss or target has been hit
Stop loss = 0.5% below entry for BUY orders.
Target = 1% above entry for BUY orders.
Add methods: open_position(), close_position(), check_exit(current_price)
```

■ Prompt 7 — The Main Bot Loop

```
Write the main trading bot loop in Python. It should:
1. Run every 60 seconds at candle close
2. Fetch latest candle using get_latest_candle()
3. Calculate indicators using add_indicators()
4. Generate signal using generate_signal()
5. If BUY signal and no open position: place buy order
6. If SELL signal or stop loss / target hit: place sell order
7. Only trade between 9:15 AM and 3:20 PM
8. Print every decision with timestamp. Add full error handling.
```

■ Prompt 8 — Safe API Key Storage

```
I do not want to hardcode my API keys in Python files.
Show me how to store my Upstox API Key, API Secret, and
Access Token in a .env file, load them using python-dotenv,
and access them with os.getenv(). Also show me how to add
.env to .gitignore so I never accidentally push keys to GitHub.
```

PART 5 — Fetch & Prepare Market Data

Before backtesting or building your live bot, you need clean historical data. This section covers pulling 6 months of 1-minute candles and preparing them correctly.

Finding Your Stock's Instrument Key

Every stock on Upstox has a unique instrument key. Here are common ones:

Stock	Instrument Key
Reliance	`NSE_EQ|INE002A01018`
TCS	`NSE_EQ|INE467B01029`
HDFC Bank	`NSE_EQ|INE040A01034`
Infosys	`NSE_EQ|INE009A01021`
NIFTY 50 Index	`NSE_INDEX|Nifty 50`
Bank NIFTY	`NSE_INDEX|Nifty Bank`

■ Claude Prompt — Fetch 6 Months of 1-Min Data

```
Write a Python script that fetches 6 months of 1-minute OHLCV
candle data from Upstox API v3. My access token is in access_token.txt.
The instrument key is [e.g. NSE_EQ|INE002A01018].
Upstox gives max 30 days per call — loop through 6 chunks automatically,
combine into one DataFrame, sort by timestamp, remove duplicates,
and save as stock_data.csv. Print the total number of rows downloaded.
Add progress messages so I know it's working.
```

Data Quality Checks — Always Do These

- Check for missing candles (gaps in timestamps) — common on low-volume stocks
- Verify OHLC logic: High must be >= Open, Close, Low; Low must be <=
- Remove rows with volume = 0 (these are illiquid periods — bad for signals)
- Check date range is complete — Upstox API sometimes skips weekends/holidays
- Sort by timestamp ascending before any analysis

■ Claude Prompt — Clean Your Data

```
I have a CSV file called stock_data.csv with columns: timestamp,
open, high, low, close, volume. Write a Python script that:
1. Checks for and removes duplicate timestamps
2. Checks for rows where volume = 0 and removes them
3. Verifies OHLC sanity (high >= low, etc.) and flags bad rows
4. Prints a summary: total rows, bad rows removed, date range
5. Saves the clean data as stock_data_clean.csv
```

PART 6 — Backtest Your Strategy

Backtesting means running your strategy on old data and seeing how it would have performed. It is not perfect — real markets have slippage, fees, and emotions — but it is the best filter before risking real money.

What a Good Backtest Script Must Include

Capital: Start with a fixed amount, e.g. ■1,00,000. Track how it changes trade by trade.

Brokerage: Deduct ■20 per trade (Zerodha/Upstox flat fee). This is ■40 per round trip.

Market Hours: Only allow trades between 9:15 AM and 3:20 PM. Square off all positions at 3:20 PM.

No Look-Ahead: Your strategy can only use data up to the current candle — NOT future candles.

Position Sizing: Define how many shares to buy. Use: capital_to_risk / entry_price.

Exit Logic: Code both stop loss AND target exits — not just entry signals.

Trade Log: Save every trade with: entry time, exit time, P&L;, reason for exit.

Equity Curve: Plot capital vs time. A rising curve = good. Flat/falling = bad strategy.

■ Master Backtest Prompt

```
I have stock_data_clean.csv with columns: timestamp, open, high, low, close, volume.
Write a complete Python backtest for this strategy:
[DESCRIBE YOUR STRATEGY — e.g. Buy when 9 EMA crosses above 21 EMA]

Rules:
- Starting capital: Rs 1,00,000
- Only trade between 9:15 AM and 3:20 PM IST
- Square off all positions at 3:20 PM no matter what
- Stop loss: 0.5% below entry price
- Target: 1% above entry price
- Brokerage: Rs 20 per trade
- One trade at a time — no pyramiding

Output required:
- Print: total trades, win rate %, total P&L in Rs, best/worst trade
- Print: max drawdown %, Sharpe ratio
- Save all trades to backtest_results.csv
- Save equity curve chart as equity_curve.png
- Use pandas, numpy, matplotlib, ta libraries
- Add beginner-friendly comments throughout
```

PART 7 — Read & Interpret Your Results

Metric	What It Means	Good Range	Red Flag
Win Rate	% of trades that were profitable	45–65%	Below 35%
Total Return	Total P&L as % of starting capital	10–30% per month	Negative
Max Drawdown	Largest peak-to-trough loss in %	Below 10%	Above 20%
Profit Factor	Total wins / Total losses	Above 1.5	Below 1.0
Sharpe Ratio	Return per unit of risk (higher = better)	Above 1.0	Below 0.5
Avg Trade P&L	Average Rs profit/loss per trade	Positive	Negative
Total Trades	Number of trades taken	50+ for validity	Below 30 — unreliable
Equity Curve	Chart of capital over time	Steadily rising	Flat or falling

How to Improve a Failing Strategy

■ Claude Prompt — Fix My Strategy

```
My backtest gave these results:
- Win rate: 34%
- Total return: -12%
- Max drawdown: 18%
- Total trades: 87

[Paste your full printed output here]

Here is my strategy code: [paste backtest.py]

What are 3 specific improvements I can make?
For each one, show me the updated code section.
```

Common Backtest Improvements

- Add a trend filter — only take buy signals when price is above the 200 EMA
- Add a volume filter — only trade when volume is above the 20-period average
- Tighten stop losses during high volatility (use ATR-based stops)
- Avoid trading in the first and last 15 minutes of the session
- Add a daily loss limit — stop trading for the day after losing 2% of capital
- Test on multiple stocks, not just one — does it work broadly?

PART 8 — Go Live: Paper Trade First

RULE: Never go live with real money until you've paper traded profitably for at least 2 weeks. Paper trading means your bot runs with real market data and real signals — but fake orders. You log what WOULD have happened without risking actual money.

Paper Trading Mode Setup

■ Claude Prompt — Paper Trading Mode

```
Convert my trading bot to paper trading mode.
Instead of placing real orders via the Upstox API,
log every trade decision to a file called paper_trades.csv.
Format: timestamp, signal, entry_price, exit_price, quantity, P&L_Rs
Show a running balance starting from Rs 1,00,000.
Print every trade as it happens with a [PAPER TRADE] prefix.
The bot should still fetch real live data and calculate real signals
— just not place real orders.
```

Go-Live Checklist

■ **Backtest:** Strategy showed positive returns over 3+ months of data

■ **Paper trade:** Paper traded live market for 2+ weeks — results match backtest roughly

■ **API tested:** Token refresh works, order placement tested with a manual Rs 1 trade

■ **Risk limits:** Max position size, daily loss limit, and stop losses coded and tested

■ **Error handling:** Bot doesn't crash on API errors — it logs and retries

■ **Square-off:** 3:20 PM auto square-off tested and confirmed working

■ **Monitoring:** You can watch the bot's log file in real time while it runs

■ **Small capital:** Start with 10–20% of planned capital for first 2 weeks live

Running Your Bot During Market Hours

■ Claude Prompt — Schedule Bot for Market Hours

```
I want my bot to run automatically at market open every day.
Write a wrapper script that:
1. Runs the token refresh script at 9:00 AM automatically
2. Starts the trading bot at 9:14 AM
3. Sends me a WhatsApp / Telegram message when the bot starts
4. Saves all logs to a file with today's date (e.g. log_2025-01-15.txt)
5. Stops the bot at 3:25 PM
Use the 'schedule' library for timing.
```

PART 9 — Risk Management & Safety Rules

This is the most important section. Most bot builders lose money not because their strategy is bad — but because they have no risk rules. These rules are non-negotiable.

Rule	What It Means	Implement as
1% Rule	Never risk more than 1% of capital on one trade	position_size = (capital * 0.01) / stop_loss_distance
Daily Loss Limit	Stop trading if you lose 2% in one day	if daily_loss > capital * 0.02: stop bot
Max Positions	Only 1–2 open positions at a time	if open_positions >= 2: no new entry
Mandatory Stop Loss	Every trade has a hard stop loss — no exceptions	Code it before placing the entry order
3:20 PM Square-off	All positions closed before market close	Scheduled job that closes everything
API Error Handling	If API fails, close position manually	Bot sends alert — you intervene
No Overnight Positions	Intraday only — no holding overnight	3:20 PM square-off ensures this
Test on Small Capital	Start with Rs 10,000 for first 2 weeks live	Scale up only after proven

■ Claude Prompt — Add All Risk Rules to My Bot

```
Add the following risk management rules to my trading bot:
1. Position sizing: Risk only 1% of current capital per trade
   (position_size = (capital * 0.01) / stop_loss_in_rupees)
2. Daily loss limit: If total day P&L drops below -2% of capital, stop all trading
3. Maximum 1 open position at any time
4. Hard stop loss: 0.5% below entry for BUY, 0.5% above for SELL
5. Mandatory 3:20 PM square-off — close all positions regardless of P&L
6. If any API call throws an exception, log the error and do NOT place order
7. Print a risk summary at the end of each day to risk_log.csv
```

PART 10 — Troubleshooting & Next Steps

Common Errors and How to Fix Them

Error	Cause	Fix
`ModuleNotFoundError`	Library not installed	pip install [library name] --break-system-packages
`Access Token Expired`	Tokens last 24 hours	Re-run your token script every morning
`Empty data / 0 rows fetched`	Wrong instrument key or date range	Check key format, verify market was open on those dates
`0 trades in backtest`	Signal conditions never met	Print indicators to check they are calculating correctly
`KeyError in DataFrame`	Column name mismatch	Print df.columns to see actual names
`Rate limit / 429 error`	Too many API calls	Add time.sleep(1) between API calls
`Order rejected`	Insufficient margin or wrong qty	Check margin in Upstox app, verify quantity is positive int
`Bot places duplicate orders`	Signal fires multiple times per candle	Add a 'position_open' flag — only enter once per signal
`Backtest too optimistic`	Overfitting to historical data	Test on out-of-sample data (different time period)

The Complete Workflow at a Glance

#	Step	Frequency
1	Open Upstox account + get API key	One time
2	Install Python, VS Code, libraries	One time
3	Learn trading basics (Part 0)	One time — but keep revising
4	Design your strategy in plain English	Per strategy
5	Use Claude prompts to build each module	Per strategy
6	Fetch 6 months of historical data	Per stock
7	Backtest and read results	Repeat until profitable
8	Paper trade for 2+ weeks	Per strategy
9	Run token script → start bot	Every market day
10	Monitor, improve, repeat	Ongoing

What to Learn Next

- Options trading basics — F&O; bots can hedge and have defined risk
- Telegram alerts — send trade notifications to your phone automatically
- Multi-stock bots — scan a universe of stocks for signals simultaneously
- Machine learning signals — use scikit-learn to predict price direction
- Cloud deployment — run your bot on AWS/GCP so it runs without your laptop
- Portfolio backtesting — backtest across 10 stocks at once with pyfolio
- Live paper trading on Zerodha Kite — best documentation and sandbox

■■ IMPORTANT DISCLAIMER

This guide is for educational purposes only. It is NOT financial advice. Trading involves significant risk of loss. Past backtest results do NOT guarantee future returns. Always paper trade before using real money. Never trade with money you cannot afford to lose. Consult a SEBI-registered financial advisor before making investment decisions.

@levelupsam • Built entirely with Claude AI • Share freely — do not sell